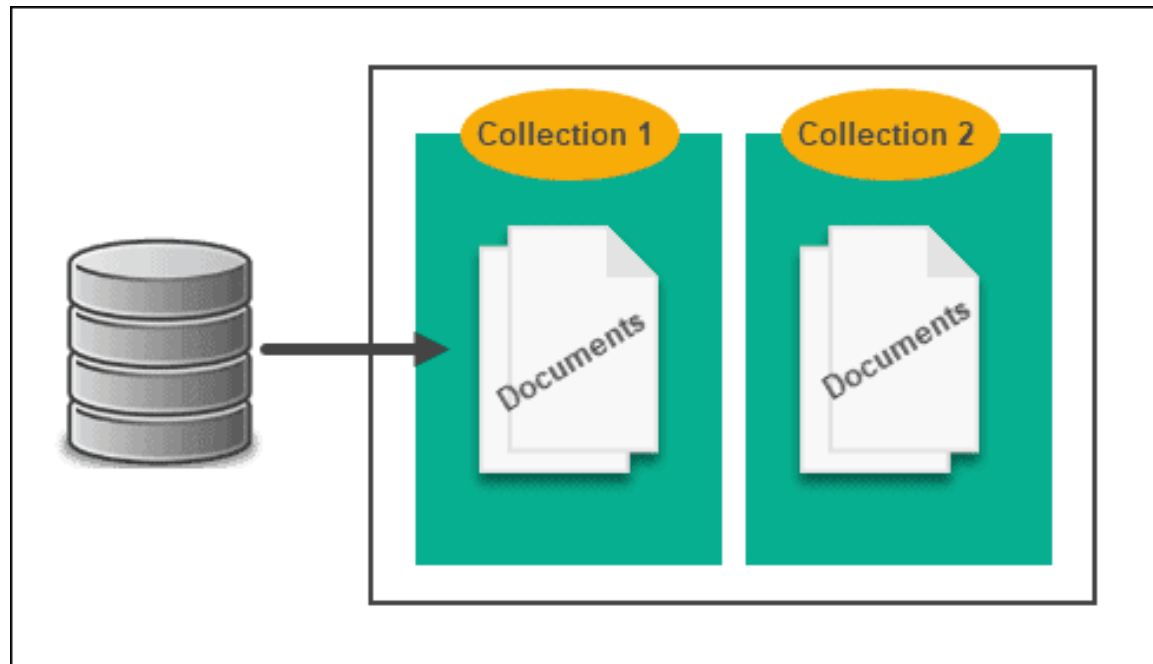


Bancos de Dados

Aula - 02

2026

SENAI - CRTI



Modelagem de Documentos em Bancos NoSQL

Unidade Curricular: Bancos de Dados

Roteiro da Aula

01

Contextualização

Situação de aprendizagem e problema real

03

Desenvolvimento

Conteúdo teórico e exemplos práticos

05

Sistematização

Resumo visual e consolidação

02

Mobilização

Ativação de conhecimentos prévios

04

Prática

Atividades durante a aula

06

Atividade Final

Modelagem em grupo + avaliação

Capacidades a Desenvolver

O que você será capaz de fazer ao final desta aula



Compreender o Modelo

Entender o modelo de dados baseado em documentos e suas vantagens em relação ao modelo relacional.



Aplicar Técnicas

Aplicar técnicas de modelagem em bancos NoSQL para resolver problemas reais de armazenamento de dados.



Identificar Estruturas

Identificar coleções e documentos, compreendendo sua organização dentro do MongoDB.



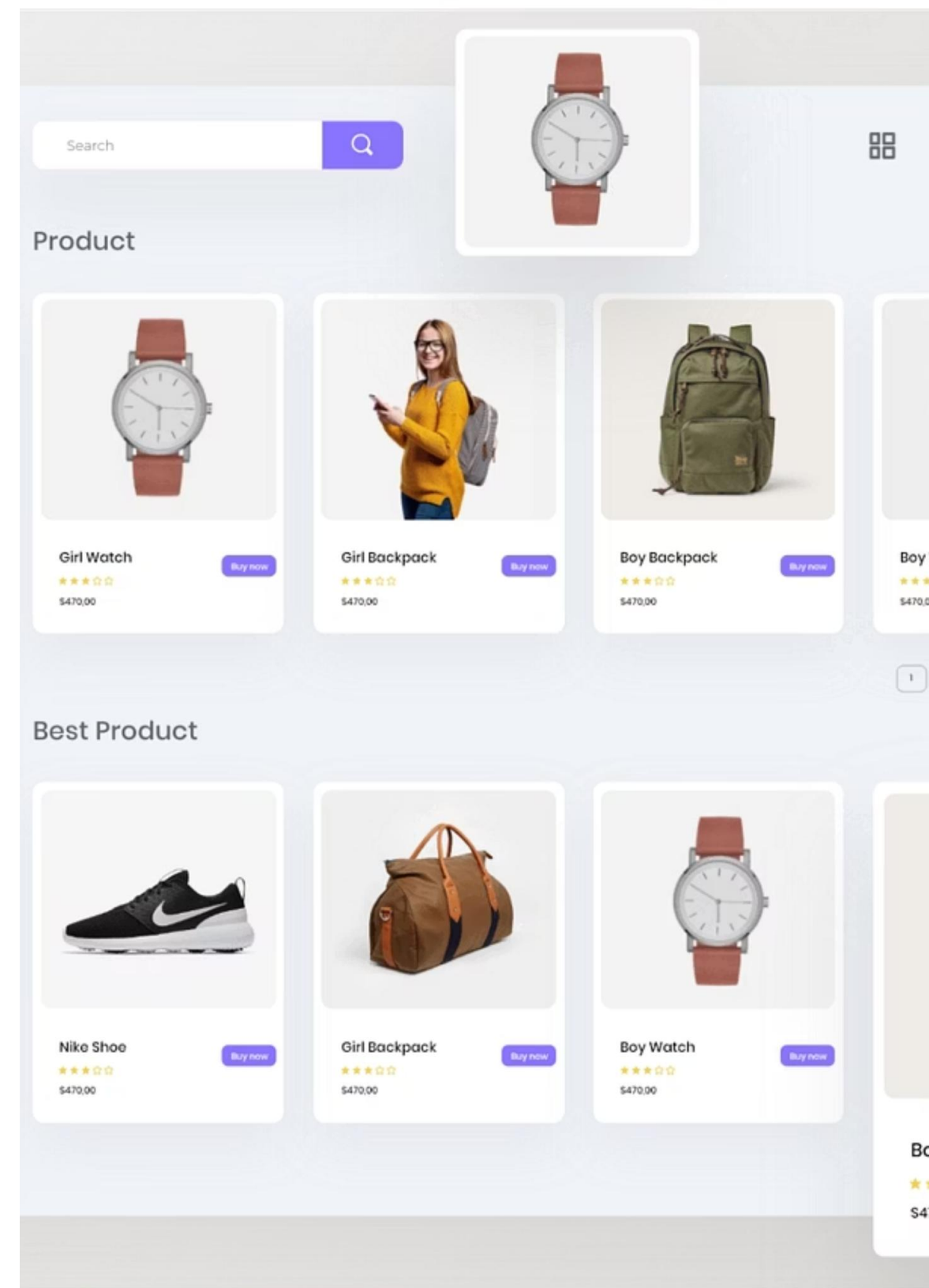
Trabalhar com JSON/BSON

Estruturar dados em formato JSON e compreender como o BSON estende essa representação.

PARTE 1 — CONTEXTUALIZAÇÃO

Situação de Aprendizagem

Imagine que você foi contratado para desenvolver o back-end de uma plataforma de e-commerce. O sistema precisa armazenar **usuários, pedidos e produtos** — mas cada produto pode ter atributos completamente diferentes (um tênis tem tamanho e cor; um notebook tem RAM e processador). No modelo relacional, isso exigiria dezenas de tabelas, JOINS complexos e alterações constantes no esquema. Como resolver?



O Problema: Rigidez do Modelo Relacional

Cenário SQL

Banco relacional tradicional para uma loja online:

- Tabela `produtos` com colunas fixas para todos os tipos
- Campos `NULL` para atributos não aplicáveis
- Tabelas auxiliares para cada variação de produto
- JOINS complexos para montar um pedido completo
- Migrations demoradas a cada nova funcionalidade

Os Desafios

- Esquema rígido dificulta adaptação rápida
- Performance cai com muitos JOINS
- Escalar horizontalmente é complexo
- Dados heterogêneos forçam gambiarras
- Mudanças de requisito geram retrabalho enorme



A necessidade de flexibilidade e escalabilidade exige uma nova abordagem de modelagem.

O que você já sabe?

Antes de entrar no conteúdo novo, vamos ativar o que já está na sua memória. Reflita sobre as perguntas abaixo e compartilhe com a turma:

? Pergunta 1

Como você representaria os dados de um usuário que pode ter zero, um ou vários endereços de entrega? Como isso ficaria em SQL?

? Pergunta 2

Toda informação armazenada em um sistema precisa ter exatamente o mesmo formato? Pense em produtos de categorias diferentes.

? Pergunta 3

O que é um JSON? Onde você já viu ou usou JSON no seu dia a dia como desenvolvedor?

? Pergunta 4

O que significa "NoSQL"? É necessariamente a ausência de SQL, ou pode significar algo mais amplo?

Modelagem Baseada em Documentos

O que é um Documento?

No contexto de bancos NoSQL orientados a documentos, um **documento** é a unidade básica de armazenamento. Ele representa uma entidade completa — como um usuário ou um pedido — em um único objeto estruturado.

- Representado no formato **JSON** (JavaScript Object Notation)
- Armazenado internamente como **BSON** (Binary JSON)
- Pode conter campos simples, objetos aninhados e arrays
- Cada documento pode ter campos diferentes — **esquema flexível**
- Identificado por um campo `_id` único gerado automaticamente

Analogia com SQL

Para quem vem do mundo relacional:

- Documento ≈ **Linha/Registro**
- Campo ≈ **Coluna**
- Coleção ≈ **Tabela**
- Banco de Dados ≈ **Banco de Dados**

A grande diferença: não há esquema fixo obrigatório!

Estrutura JSON: A Base de Tudo

JSON (JavaScript Object Notation) é um formato leve de troca de dados, fácil de ler por humanos e de processar por máquinas. É a linguagem com a qual você se comunica com o MongoDB.

Tipos de Dados em JSON

- **String:** "João Silva"
- **Number:** 25 ou 99.90
- **Boolean:** true ou false
- **Null:** null
- **Array:** ["tecnologia", "música"]
- **Object:** {"cidade": "São Paulo"}

Exemplo de Documento JSON

```
{
  "nome": "João",
  "idade": 25,
  "endereco": {
    "cidade": "São Paulo",
    "estado": "SP"
  },
  "interesses": [
    "tecnologia",
    "programação"
  ]
}
```

BSON: JSON Turbinado para o MongoDB

O que é BSON?

BSON (Binary JSON) é a representação **binária e serializada** do JSON usada internamente pelo MongoDB para armazenamento e transferência de dados em rede.

Não é o que você escreve — é o que o MongoDB usa por baixo dos panos, de forma transparente para o desenvolvedor.

Vantagens do BSON sobre JSON

- **Tipos adicionais:** Date, ObjectId, Binary, Decimal128
- **Eficiência:** parsing mais rápido por ser binário
- **Precisão:** suporte nativo a inteiros de 64 bits
- **Traversal:** permite percorrer documentos sem parsear tudo
- **Tamanho:** geralmente mais compacto para dados numéricos



Para você, desenvolvedor Node.js, o driver do MongoDB converte JSON ↔ BSON automaticamente.

Exemplo Real: Documento de Usuário no MongoDB

Veja como um documento completo de usuário seria representado no MongoDB, incluindo dados aninhados e arrays:

```
{
  "_id": ObjectId("64a1f2c3e4b05a1234567890"),
  "nome": "João Silva",
  "email": "joao@email.com",
  "idade": 25,
  "ativo": true,
  "dataCadastro": ISODate("2024-01-15"),
  "endereco": {
    "rua": "Av. Paulista, 1000",
    "cidade": "São Paulo",
    "estado": "SP",
    "cep": "01310-100"
  },
  "interesses": ["tecnologia", "programação", "games"],
  "ultimoAcesso": ISODate("2024-06-10")
}
```

📌 Observe: `_id` é gerado automaticamente pelo MongoDB. O campo `endereco` é um **objeto aninhado** (subdocumento) e `interesses` é um **array**.



Coleções: Agrupando Documentos

Uma **coleção** é um agrupamento de documentos dentro de um banco de dados MongoDB. É o equivalente a uma tabela no modelo relacional — mas com uma diferença fundamental: **não há esquema obrigatório**. Documentos dentro de uma mesma coleção podem ter campos completamente diferentes. O MongoDB não impõe nem valida a estrutura (a menos que você configure isso explicitamente com JSON Schema Validation).

Coleções em um Sistema de E-commerce

Em uma aplicação real de e-commerce, as coleções organizam os diferentes tipos de dados. Veja como ficaria a estrutura:



usuarios

Dados de clientes: nome, email, endereços, histórico de compras e preferências.



pedidos

Pedidos realizados: itens, valores, status, forma de pagamento e rastreamento.



produtos

Catálogo de produtos: nome, categoria, preço, atributos específicos e estoque.



avaliacoes

Avaliações de clientes: nota, comentário, data e referência ao produto e usuário.

Flexibilidade: O Superpoder do Esquema Dinâmico

No MongoDB, documentos da **mesma coleção** podem ter estruturas completamente diferentes. Isso resolve o problema de produtos com atributos heterogêneos:

Produto: Tênis

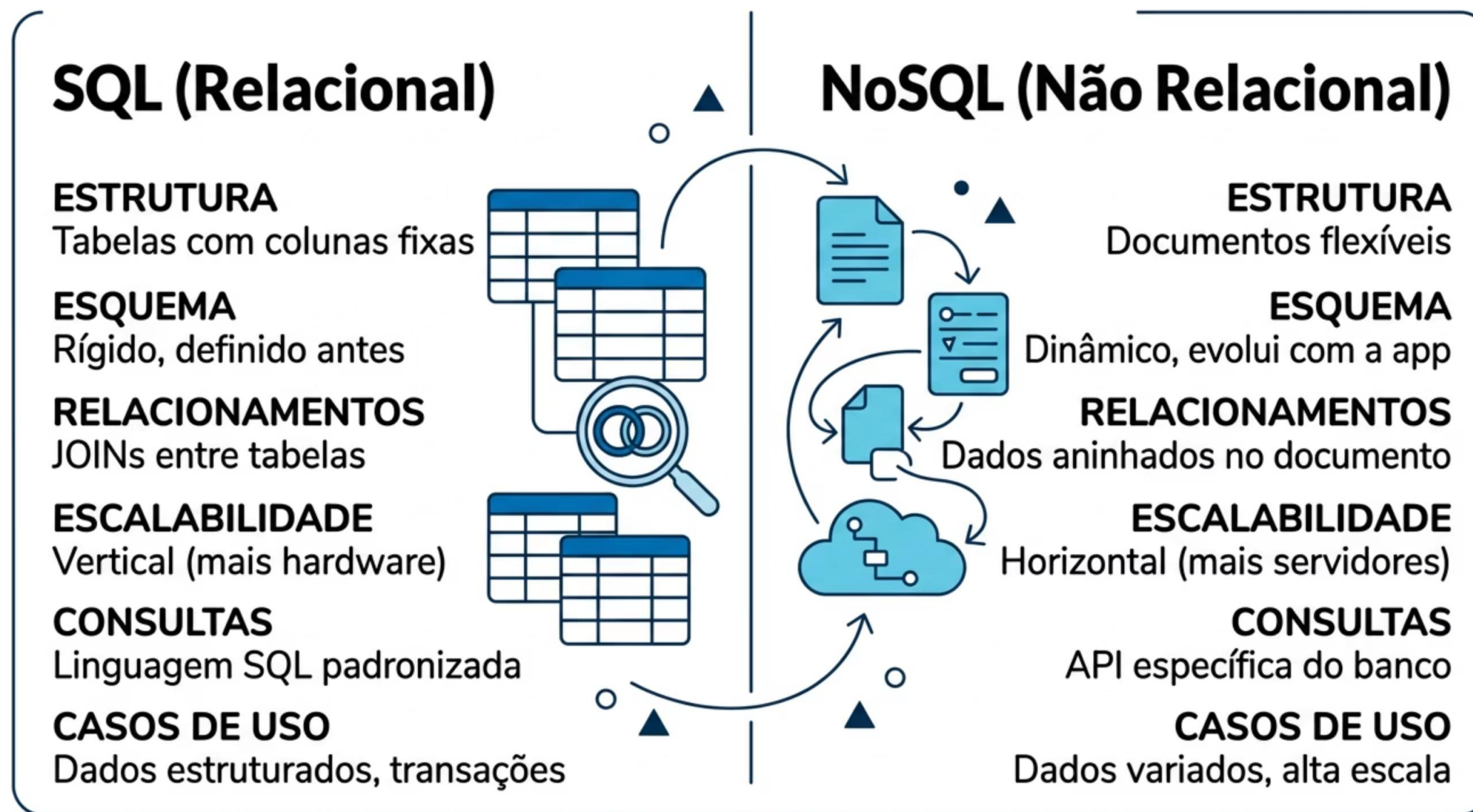
```
{
  "_id": ObjectId("..."),
  "nome": "Tênis Runner Pro",
  "categoria": "calçados",
  "preco": 299.90,
  "tamanhos": [38, 39, 40, 41],
  "cores": ["preto", "branco"],
  "material": "mesh respiravel"
}
```

Produto: Notebook

```
{
  "_id": ObjectId("..."),
  "nome": "Notebook UltraBook X1",
  "categoria": "informatica",
  "preco": 4599.00,
  "ram": "16GB",
  "processador": "Intel i7-12th",
  "armazenamento": "512GB SSD",
  "tela": "15.6 polegadas"
}
```

✅ Ambos na coleção **produtos** — sem tabelas auxiliares, sem campos NULL desnecessários!

SQL vs NoSQL: Comparação de Modelagem



Embedding vs Referência: Quando Usar Cada Abordagem

Uma das decisões mais importantes na modelagem NoSQL é: **embutir os dados** no mesmo documento ou **referenciar** outro documento?

✓ Embedding (Dados Aninhados)

Use quando:

- Os dados são sempre acessados juntos
- Há relacionamento 1:1 ou 1:N com poucos itens
- Os dados não mudam com frequência independentemente

Exemplo: endereço dentro do documento do usuário

🔗 Referência (ID de Outro Documento)

Use quando:

- O dado é compartilhado entre vários documentos
- Há relacionamento N:N ou 1:N com muitos itens
- Os dados crescem muito ou mudam independentemente

Exemplo: ID do produto dentro do pedido

Exemplo: Modelagem de Pedido com Embedding

Veja como um pedido completo pode ser modelado em um **único documento**, evitando JOINS e tornando a leitura muito mais eficiente:

```
{
  "_id": ObjectId("64b2e3f1..."),
  "numero": "PED-2024-001",
  "cliente": {
    "nome": "Maria Souza",
    "email": "maria@email.com"
  },
  "itens": [
    { "produto": "Tênis Runner", "quantidade": 1, "preco": 299.90 },
    { "produto": "Meia Esportiva", "quantidade": 3, "preco": 19.90 }
  ],
  "total": 359.60,
  "status": "entregue",
  "dataPedido": ISODate("2024-06-01")
}
```

✔ Uma única leitura retorna o pedido completo — sem JOIN com tabela de clientes ou de itens!

Criando Seus Primeiros Documentos JSON

Chegou a hora de colocar a mão na massa! Individualmente ou em dupla, crie documentos JSON para os seguintes cenários:

1

Aluno de Escola

Crie um documento representando um aluno. Inclua: nome, matrícula, data de nascimento, curso, disciplinas cursadas (array com nome e nota) e endereço (objeto aninhado).

2

Produto de Loja

Crie um documento para um produto de sua escolha. Lembre-se: inclua atributos específicos da categoria, preço, estoque e pelo menos um array de dados.

3

Post de Rede Social

Modele um post com: texto, autor, data, array de curtidas (usuários), array de comentários (cada um com autor e texto).



Tempo estimado: 15 minutos. Use papel, editor de texto ou VS Code para escrever o JSON.

Identificando Coleções em um Sistema

Em grupos de 3 a 4 pessoas, escolha um dos sistemas abaixo e **identifique todas as coleções necessárias**. Para cada coleção, liste os principais campos e decida o que seria embutido (embedded) e o que seria referenciado:

🏠 Sistema Escolar

Alunos, professores, turmas, disciplinas, notas, frequência.

🛒 Marketplace

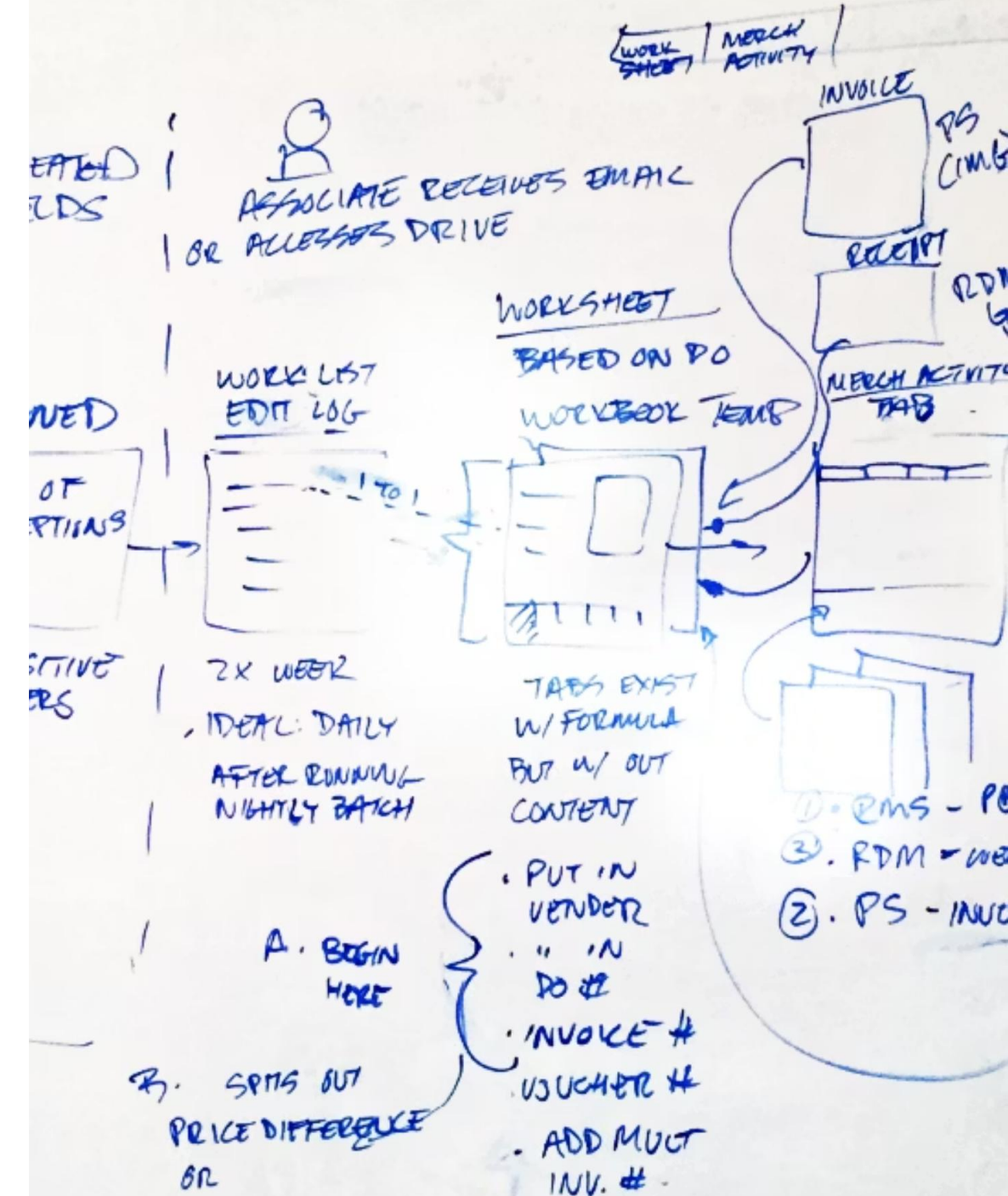
Vendedores, compradores, produtos, pedidos, avaliações, categorias.

📅 App de Tarefas

Usuários, projetos, tarefas, comentários, etiquetas, histórico.

🎵 Streaming de Música

Usuários, artistas, álbuns, músicas, playlists, histórico de plays.



SQL vs NoSQL: Comparando na Prática

Dada a situação abaixo, modele os dados das **duas formas** — primeiro em SQL (tabelas e colunas), depois em NoSQL (documentos JSON) — e depois compare as diferenças:

- 📄 **Cenário:** Um sistema de blog com usuários que escrevem posts. Cada post pode ter vários comentários. Cada comentário pertence a um usuário. Os posts têm tags (palavras-chave).

Modelagem SQL

Crie o diagrama de tabelas com:

- Tabela `usuarios`
- Tabela `posts`
- Tabela `comentarios`
- Tabela `tags` + tabela de associação
- Chaves estrangeiras entre elas

Modelagem NoSQL

Crie documentos JSON com:

- Coleção `usuarios`
- Coleção `posts` com comentários embutidos
- Tags como array dentro do post
- Referência ao autor por ID

🕒 ⌚ Tempo estimado: 20 minutos. Discuta com seu grupo: qual abordagem é mais natural para consultas de leitura?



BSON

Boas Práticas na Modelagem de Documentos

- Modele para as Consultas, não para os Dados
A principal regra do MongoDB: organize os dados da forma como você vai **consultá-los**, não da forma como eles existem no mundo real.
- Evite Documentos Muito Grandes
O MongoDB tem um limite de **16MB por documento**. Arrays que crescem indefinidamente (ex: todos os comentários de um post viral) são um anti-pattern.
- Use Nomes de Campos Consistentes
Como o esquema é flexível, adote uma convenção (`camelCase` ou `snake_case`) e aplique em toda a aplicação para facilitar manutenção.
- Crie Índices para Campos de Consulta
Campos usados em filtros (`find`) e ordenações (`sort`) devem ter índices criados para garantir performance.

Anti-patterns: O que Evitar no MongoDB

✗ Arrays de Referências Gigantes

Evite guardar centenas ou milhares de IDs em um array dentro de um documento. Isso impacta performance e viola o limite de 16MB.

✗ Pensar em JOINS como no SQL

O `$lookup` do MongoDB existe, mas usar muitos `$lookup` indica que o modelo pode ser melhorado com embedding estratégico.

✗ Coleção Única para Tudo

Misturar tipos muito diferentes de documentos em uma coleção só porque "são todos dados" dificulta consultas e manutenção.

✗ Ignorar Validação de Esquema

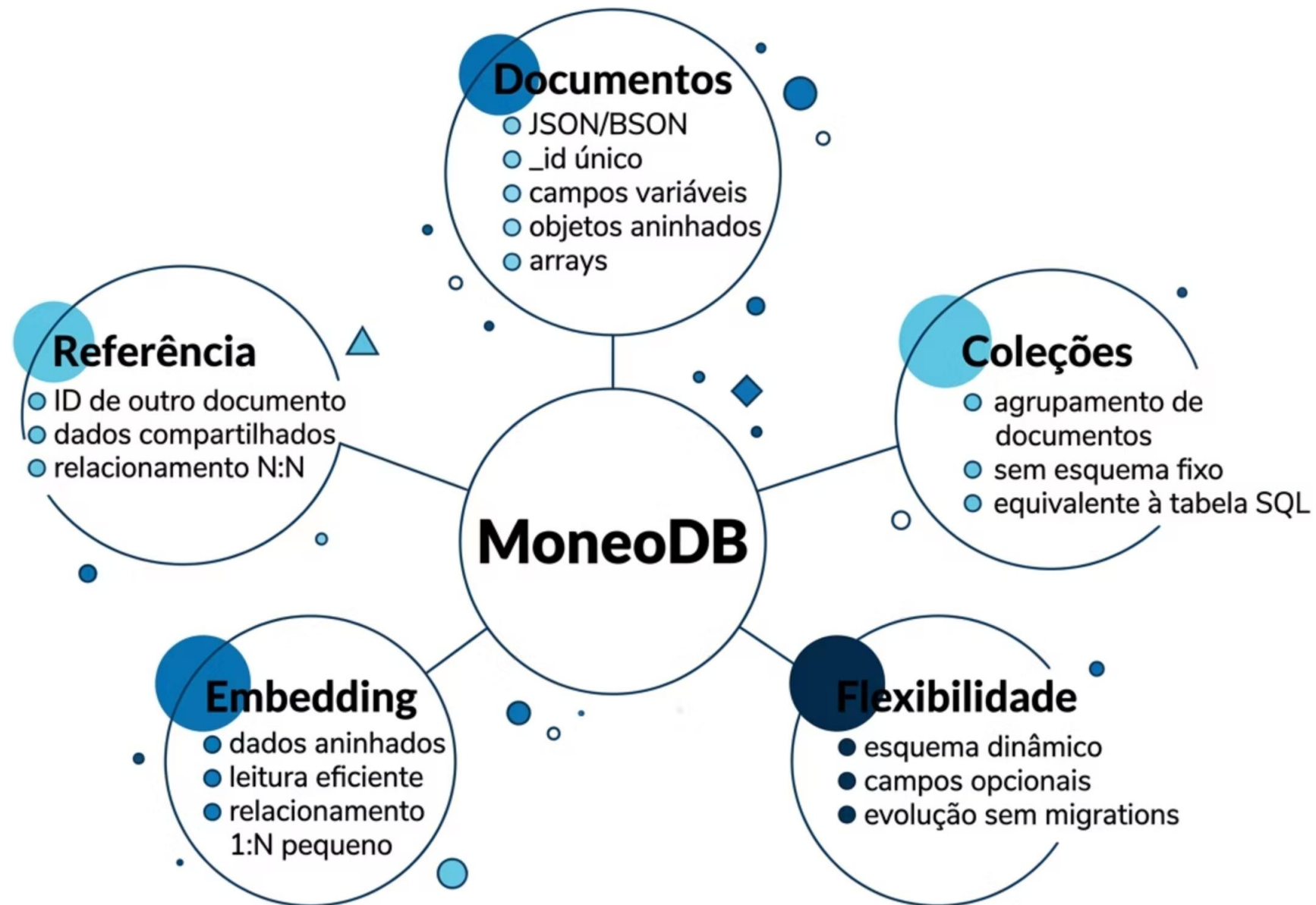
Flexibilidade não significa caos. Use JSON Schema Validation do MongoDB para garantir integridade mínima dos dados em produção.

Resumo Visual: SQL vs NoSQL

Consolidando os conceitos aprendidos em uma visão comparativa clara:

Conceito	Modelo Relacional (SQL)	Modelo de Documentos (NoSQL)
Unidade de dados	Linha/Registro	Documento (JSON/BSON)
Agrupamento	Tabela	Coleção
Esquema	Rígido, definido previamente	Dinâmico, flexível
Relacionamentos	JOINS com chaves estrangeiras	Embedding ou referência por ID
Tipos de dados	Tipos SQL padrão	JSON + tipos BSON estendidos
Escalabilidade	Principalmente vertical	Horizontal (sharding)
Exemplo de banco	MySQL, PostgreSQL, Oracle	MongoDB, CouchDB, Firestore

Mapa Mental: Conceitos Fundamentais



PARTE 5 — ATIVIDADE FINAL (30 MIN)

Projeto em Grupo: Modelagem Completa com MongoDB

Em grupos de 3 a 5 pessoas, vocês irão **modelar os dados de um sistema completo** utilizando MongoDB. Esta atividade simula o trabalho real de um desenvolvedor back-end ao iniciar um novo projeto.



Instruções da Atividade Final

1

Escolha um Sistema

Loja virtual, rede social, app de tarefas, sistema escolar, app de delivery, streaming de filmes — ou proponha seu próprio!

2

Defina as Coleções

Identifique **pelo menos 2 coleções**. Liste os campos de cada uma e justifique por que são coleções separadas.

3

Crie Documentos de Exemplo

Para cada coleção, escreva **pelo menos 2 documentos JSON** com dados reais. Use objetos aninhados e arrays.

4

Decida Embedding vs Referência

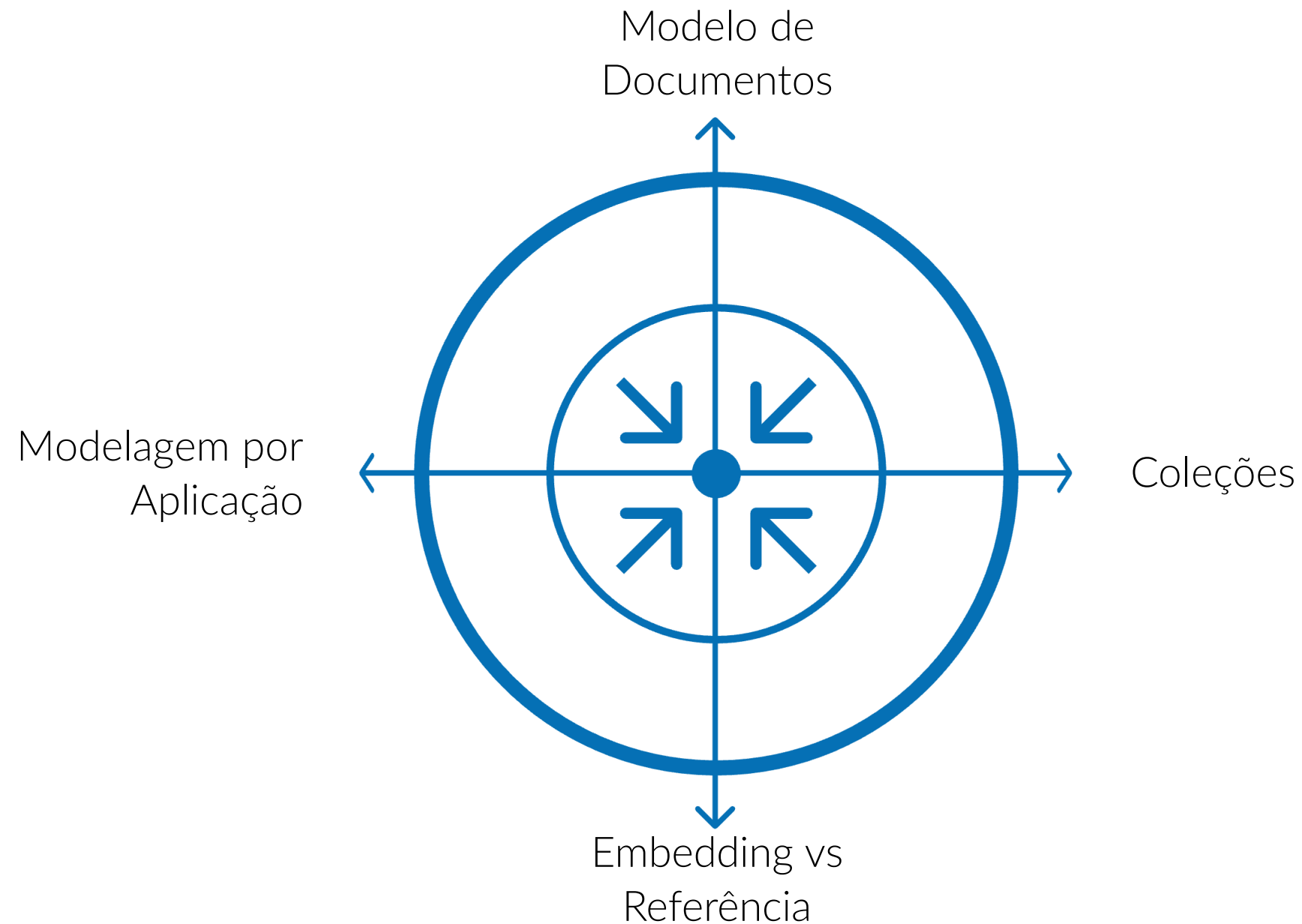
Para cada relacionamento entre coleções, justifique se usaria **embedding** ou **referência por ID**.

5

Apresente para a Turma

Cada grupo terá **5 minutos** para apresentar a modelagem, explicar as decisões e responder perguntas.

Recapitulando: O que Aprendemos Hoje



A modelagem de documentos representa uma mudança de mentalidade: ao invés de normalizar dados em tabelas, organizamos as informações da forma como a aplicação as consome. Isso resulta em código mais simples, consultas mais rápidas e maior flexibilidade para evoluir o sistema.

Recursos para Aprofundamento

Documentação Oficial

- **MongoDB Docs:** docs.mongodb.com — guia completo, tutoriais e referência da API
- **MongoDB University:** university.mongodb.com — cursos gratuitos com certificação
- **BSON Specification:** bsonspec.org — especificação técnica do formato BSON

Ferramentas Recomendadas

- **MongoDB Compass:** GUI para explorar e consultar dados visualmente
- **mongosh:** shell moderno para interagir via terminal
- **Mongoose:** ODM para Node.js com validação e esquemas
- **MongoDB Atlas:** banco na nuvem gratuito para estudos

✔ 💡 **Dica:** O MongoDB Atlas oferece um cluster gratuito (M0 Sandbox) — crie sua conta e pratique online sem instalar nada!

